

UNIVERSITÉ DE NOUAKCHOTT
Faculté des Sciences et Techniques
Master 1 Intelligence Artificielle

Méthodes d'Ensemble & Fondements des Réseaux Profonds

Advanced Machine Learning (AML) — Advanced Deep Learning (ADL)

Présenté par :

Mohamed Salem Ebnou Oubeid — C34613

Fatimata Issa Saw — C21304

Oussama Sid'Ahmed Hedy — C34603

Encadreur : Dr. Rivea Sadegh

Date de soutenance : 13 juin 2026

1. Introduction

Ce rapport présente les travaux réalisés dans le cadre du module Advanced Machine Learning (AML) et Advanced Deep Learning (ADL) — Paire 1. Il couvre deux axes complémentaires : les **méthodes d'ensemble** en Machine Learning classique, et les **fondements mathématiques des réseaux de neurones profonds**.

Ces deux domaines, souvent présentés séparément, partagent un lien fondamental : la technique du **Dropout**, omniprésente en Deep Learning, peut être interprétée comme une forme implicite de Bagging. Ce rapport explore ce pont conceptuel et présente les implémentations associées.

2. Méthodes d'Ensemble (AML)

2.1 Décomposition Biais-Variance

L'erreur quadratique moyenne d'un estimateur se décompose en trois termes fondamentaux :

$$E[(y - \hat{y})^2] = \text{Biais}^2 + \text{Variance} + \sigma^2$$

Le **biais** représente l'erreur systématique due à un modèle trop simple (sous-apprentissage). La **variance** mesure la sensibilité du modèle aux fluctuations des données d'entraînement (sur-apprentissage). Le bruit irréductible σ^2 est inhérent aux données.

Les méthodes d'ensemble visent à réduire l'un ou l'autre de ces termes : le Bagging réduit la variance, le Boosting réduit le biais.

2.2 Théorème de Condorcet

Ce théorème garantit que le vote majoritaire de N classifieurs indépendants, chacun correct avec probabilité $p > 0,5$, est supérieur à un classifieur unique. La probabilité d'une décision correcte vaut :

$$P(\text{majorité correcte}) = \sum_{k=\lfloor N/2 \rfloor + 1}^N C(N, k) \cdot p^k \cdot (1-p)^{N-k} \text{ pour } k \text{ de } \lfloor N/2 \rfloor + 1 \text{ à } N$$

À mesure que $N \rightarrow \infty$, cette probabilité tend vers 1, indépendamment de p (tant que $p > 0,5$). Notre simulation Monte Carlo avec 50 000 itérations confirme ce résultat théorique à $\pm 0,001$ près.

2.3 Bagging, Boosting et Stacking

Méthode	Principe	Réduction de	Parallélisable
Bagging	Bootstrap + agrégation	Variance	Oui
R. Forest	Bagging + features aléatoires	Variance	Oui
AdaBoost	Poids adaptatifs séquentiels	Biais	Non
Grad. Boosting	Descente de gradient fct.	Biais	Non
XGBoost	GB + régularisation L1/L2	Biais	Partiel
Stacking	Méta-apprenant	Biais + Var.	Oui (niv. 1)

Nos expériences sur le dataset Digits (1 797 images, 10 classes) montrent que Random Forest (100 arbres) atteint 0,972 d'accuracy contre 0,873 pour un arbre seul, confirmant l'apport du Bagging.

XGBoost atteint 0,979 avec un temps d'entraînement compétitif.

3. Fondements des Réseaux Profonds (ADL)

3.1 Du Perceptron au MLP

Le perceptron de Rosenblatt (1958) est le premier modèle d'apprentissage automatique neuronal. Il ne peut résoudre que des problèmes linéairement séparables, ce qui le rend incapable de traiter le XOR (Minsky & Papert, 1969). L'ajout de couches cachées, formant un **Multi-Layer Perceptron (MLP)**, surmonte cette limitation.

Un MLP à L couches calcule successivement :

$$h^{(l)} = f(W^{(l)} \cdot h^{(l-1)} + b^{(l)})$$

où f est une fonction d'activation non-linéaire. La sortie finale utilise la fonction Softmax pour la classification multi-classes.

3.2 Backpropagation

L'algorithme de rétropropagation (Rumelhart, Hinton & Williams, 1986) calcule le gradient de la fonction de coût par rapport à chaque paramètre en appliquant la règle de la chaîne de la couche de sortie vers l'entrée :

$$\delta^{(l)} = (W^{(l+1)} \cdot \delta^{(l+1)}) \cdot f'(z^{(l)})$$

$$\partial L / \partial W^{(l)} = \delta^{(l)} \cdot (h^{(l-1)}) \text{ puis } W^{(l)} \leftarrow W^{(l)} - \eta \cdot \partial L / \partial W^{(l)}$$

Notre implémentation from scratch en NumPy reproduit fidèlement cet algorithme et atteint 0,952 d'accuracy sur Digits après 100 époques de mini-batch SGD.

3.3 Fonctions d'activation

Fonction	Formule	Dérivée	Usage
Sigmoid	$\sigma(x) = 1/(1+e^{-x})$	$\sigma \cdot (1-\sigma)$	Sortie binaire
Tanh	$\tanh(x)$	$1 - \tanh^2(x)$	LSTM, RNN
ReLU	$\max(0, x)$	1 si $x > 0$, 0 sinon	Défaut (caché)
GELU	$x \cdot \Phi(x)$	complexe	Transformers

ReLU est préférée car sa dérivée vaut 1 pour $x > 0$, ce qui atténue le problème du *vanishing gradient* — un problème critique avec Sigmoid et Tanh dont la dérivée est bornée par 0,25.

3.4 Optimiseurs

La descente de gradient stochastique (SGD) met à jour les poids sur des mini-batches. Des variantes améliorent la convergence : Momentum accumule les gradients passés ; **Adam** (Kingma & Ba, 2015) adapte le taux d'apprentissage par paramètre :

$$m_t = \beta \cdot m_{t-1} + (1-\beta) \cdot g_t \quad v_t = \beta \cdot v_{t-1} + (1-\beta) \cdot g_t^2$$

$$\theta_{t+1} = \theta_t - \eta \cdot m_t / (\sqrt{v_t} + \epsilon) \text{ avec } \beta = 0,9, \beta_2 = 0,999$$

4. Pont ML ↔ DL : Dropout = Bagging Implicite

Le **Dropout** (Srivastava et al., 2014) désactive aléatoirement chaque neurone avec probabilité p lors de l'entraînement. Gal & Ghahramani (ICML 2016) ont démontré formellement que :

Critère	Bagging (explicite)	Dropout (implicite)
Nb. sous-modèles	T modèles indépendants	2^M sous-réseaux
Poids	Indépendants	Partagés
Entraînement	Parallèle	Un masque par batch
Inférence	Vote / Moyenne	Poids $\times (1-p)$
Coût mémoire	$O(T)$	$O(1)$

Cette équivalence est profonde : le Dropout entraîne implicitement un ensemble de 2^M réseaux partageant leurs poids. À l'inférence, multiplier les activations par $(1-p)$ approxime la moyenne de cet ensemble. Cette interprétation permet également d'utiliser le Dropout à l'inférence pour estimer l'incertitude du modèle (*MC-Dropout*).

5. Conclusion

Ce rapport a présenté les méthodes d'ensemble et les fondements des réseaux profonds en mettant en évidence leur complémentarité. Le Bagging et le Boosting offrent des solutions robustes pour les données tabulaires, tandis que le MLP avec backpropagation constitue le socle du Deep Learning moderne.

Le lien Dropout = Bagging illustre que ces deux domaines ne sont pas disjoints : les principes mathématiques fondamentaux — réduction de variance par agrégation — transcendent la frontière ML/DL. Ce pont conceptuel motive directement notre pipeline hybride présenté dans le rapport de projet.

Bibliographie

- [1] Géron, A. (2023). *Hands-On Machine Learning* (3e éd.). O'Reilly.
- [2] Goodfellow, I., Bengio, Y. & Courville, A. (2016). *Deep Learning*. MIT Press.
- [3] Dietterich, T. G. (2000). Ensemble Methods in Machine Learning. *MCS 2000*.
- [4] Rumelhart, D. E., Hinton, G. E. & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536.
- [5] Srivastava, N. et al. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *JMLR*, 15, 1929–1958.
- [6] Gal, Y. & Ghahramani, Z. (2016). Dropout as a Bayesian Approximation. *ICML 2016*.
- [7] Kingma, D. P. & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *ICLR 2015*.
- [8] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.